UNIVERSITÄT
DES
SAARLANDES

PHOTON BEAM SPLATTING FOR PARTICIPATING MEDIA

Master Thesis

by

Honglu Ma

Advisor:
Ömercan Yazici

1. Referee:
Prof. Dr.-Ing. Philipp Slusallek

August 7, 2026

Contents

Sworn declaration

I declare under oath that I have prepared the paper at hand independently and without the help of others and that I have not used any other sources and resources than the ones stated. Parts that have been taken literally or correspondingly from published or unpublished texts or other sources have been labelled as such. This paper has not been presented to any examination board in the same or similar form before.

Saarbrücken, _____ _____

Chapter I.

Introduction

Chapter II.

Background

To simulate a physical phenomenon — in our case, light — on a computer, one must first model it mathematically. This chapter assembles that model piece by piece. We begin with radiometry, the vocabulary in which light is measured, then state the rendering equation that lies behind every physically based renderer and extend it to participating media. Since these equations admit no closed-form solution for realistic scenes, we introduce Monte Carlo integration, the standard machinery for estimating them, together with the sampling techniques specific to media. We close with the two computational paradigms of computer graphics, ray tracing and rasterization, and how they are employed differently; our method, presented in Chapter IV, is a hybrid of both.

1. Radiometry

The colors we see are the result of stimulated photoreceptors on the retina. The receptors known as *cones* (responsible for vision in the higher intensity range, in contrast to the *rods*) respond differently to light of different wavelengths. Light emitted from a source bounces off the surrounding surfaces; along the way, some wavelengths are absorbed more strongly than others, and the light that finally lands on the receptors is processed by the brain into a sensation of color, its intensity into one of brightness. To simulate vision faithfully, a renderer must therefore keep track of how much light energy arrives at a sensor, from which directions, and at which wavelengths, after all the scattering and absorption in the scene. Radiometry provides the quantities for this bookkeeping; we introduce them from the coarsest to the finest, following the treatment in [7].

Light carries energy, measured in Joules (J). *Radiant flux* (or *power*), denoted by the capital Greek letter Φ, is the amount of energy emitted, transported, or

received per unit time,

$$\Phi = \frac{\mathrm{d}Q}{\mathrm{d}t}, \tag{II.1}$$

measured in Watts (W = J/s). Flux describes a light source or a receiving surface *as a whole*, e.g. the total power emitted by a light bulb, with no notion of where on the surface or from which direction the energy flows.

Refining flux over area gives *irradiance*: the flux received per unit surface area,

$$E(\mathbf{x}) = \frac{\mathrm{d}\Phi}{\mathrm{d}A(\mathbf{x})}, \tag{II.2}$$

measured in W/m^2. The same quantity measured for light *leaving* a surface is called *radiant exitance*.

Before refining over direction as well, we need a way to measure sets of directions. A direction can be identified with a point on the unit sphere, and a set of directions with a region on it; the *solid angle* of the set is the area of that region, measured in steradians (sr). It is the three-dimensional analogue of an angle in the plane, which measures a set of directions by the area it covers on the unit sphere: the full sphere subtends a solid angle of 4π, a hemisphere 2π. A differential solid angle $\mathrm{d}\omega$ then stands for a single direction ω together with its infinitesimal neighborhood.

The quantity most relevant to rendering is *radiance*: the flux arriving at (or leaving) a point of a surface, per unit projected area, per unit solid angle,

$$L(\mathbf{x}, \omega) = \frac{\mathrm{d}^2\Phi}{\mathrm{d}\omega\, \mathrm{d}A^{\perp}(\mathbf{x})} = \frac{\mathrm{d}^2\Phi}{\mathrm{d}\omega\, \mathrm{d}A(\mathbf{x}) \cos\theta}, \tag{II.3}$$

measured in W/m^2/sr. Here $\mathrm{d}A^{\perp} = \mathrm{d}A \cos\theta$ is the surface area projected perpendicular to ω, with θ the angle between ω and the surface normal: a surface viewed at a grazing angle intercepts less light per unit of its own area, and the projection makes radiance a property of the light ray itself rather than of the receiving surface. Indeed, radiance remains constant along a ray in empty space — a property we will lose, deliberately, once we enter participating media in Section 3. Radiance is exactly what a pinhole camera measures: the light energy arriving at a specific position on the film from the single direction passing through the pinhole (at a certain time, assuming we render a static image rather than a video). Conversely, integrating radiance over the hemisphere of incoming directions recovers irradiance, $E(\mathbf{x}) = \int L(\mathbf{x}, \omega) \cos\theta \, \mathrm{d}\omega$.

All these quantities are, strictly speaking, functions of wavelength. As is common practice, we work with three representative wavelength bands — the RGB channels — and every radiometric quantity in this thesis should be read

as a triple. Now that we have the basic quantities, we can model how light interacts with surfaces.

2. The Rendering Equation and Global Illumination

Light emitted from a source is absorbed and scattered by the solid surfaces of a scene, often many times, before a fraction of it finally reaches the eye. At every such surface interaction we wish to know the relationship between the incoming and the outgoing light: given the light arriving at a surface point from all directions, what is the radiance leaving toward one particular direction? With such a relation we can follow a light ray from the source step by step, compute its changes along the trip, and hope that it ends up in the eye — or, in reverse, trace from the eye and hope to reach a light source (more on this in Section 4). This relation is the *rendering equation*, introduced by Kajiya [5]:

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\mathcal{H}^2} f_r(\mathbf{x}, \omega_i, \omega_o)\, L_i(\mathbf{x}, \omega_i) \cos\theta_i \, \mathrm{d}\omega_i \,. \tag{II.4}$$

It states that the outgoing radiance L_o at surface point $\mathbf{x}$ in direction ω_o is the sum of two contributions. The first, $L_e(\mathbf{x}, \omega_o)$, is the radiance emitted by the surface itself and is nonzero only for light sources. The second collects, over the unit hemisphere $\mathcal{H}^2$ above the surface, the incoming radiance $L_i(\mathbf{x}, \omega_i)$ from every direction ω_i, weighted by two factors: the cosine of the angle θ_i between ω_i and the surface normal, which accounts for the foreshortening of the surface as seen from ω_i (the projected-area term already familiar from the definition of radiance), and the function f_r, which describes the material.

This function $f_r(\mathbf{x}, \omega_i, \omega_o)$ is the *bidirectional reflectance distribution function* (BRDF). Light arriving from an infinitesimal cone of directions $\mathrm{d}\omega_i$ around ω_i alone deposits the differential irradiance $\mathrm{d}E(\omega_i) = L_i(\mathbf{x}, \omega_i) \cos\theta_i \, \mathrm{d}\omega_i$ — precisely one integrand slice of the hemispherical integral $E = \int L \cos\theta \, \mathrm{d}\omega$ from Section 1. The BRDF is defined as the ratio of the differential outgoing radiance this deposit causes toward ω_o to the deposit itself,

$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{\mathrm{d}L_o(\mathbf{x}, \omega_o)}{\mathrm{d}E(\mathbf{x}, \omega_i)} = \frac{\mathrm{d}L_o(\mathbf{x}, \omega_o)}{L_i(\mathbf{x}, \omega_i) \cos\theta_i \, \mathrm{d}\omega_i} \,, \tag{II.5}$$

with unit sr^{-1}; summing the resulting contributions $\mathrm{d}L_o = f_r L_i \cos\theta_i \, \mathrm{d}\omega_i$ over all incoming directions is exactly what the integral in Eq. (II.4) does. The BRDF is where the appearance of a material lives. Physically plausible BRDFs are reciprocal — swapping ω_i and ω_o leaves the value unchanged — and energy

conserving, reflecting in total no more energy than they receive. Two idealized cases bound the spectrum of real materials. An ideal diffuse (Lambertian) surface reflects incoming light equally in all directions, giving the constant BRDF $f_r = \rho/\pi$ with albedo ρ. An ideal specular surface reflects each incoming direction into exactly one outgoing direction, so its BRDF is a Dirac delta distribution: a mirror reflects ω_i about the normal, while a transparent surface additionally bends light *into* the material according to Snell's law of refraction, the Fresnel equations dictating how the energy splits between the reflected and refracted parts. Once transmission is included, light may leave through the lower hemisphere as well; the generalization of the BRDF to the full sphere of directions is called the *bidirectional scattering distribution function* (BSDF), and the integral in Eq. (II.4) then runs over the whole sphere. Specular surfaces are what focus light into the bright, curved patterns known as *caustics* — a phenomenon that is notoriously difficult for eye-path tracing and a key motivation for the photon-based methods discussed in Chapter III.

One may notice that the incoming radiance L_i inside the integral is itself the outgoing radiance of some other surface point — the rendering equation is recursive, and as written it describes only a single scattering event of light's whole journey from a source to the eye. The effort of estimating the entirety of all such journeys is called solving the *global illumination* problem. In the early age of computer graphics, when computing power was scarce, renderers avoided it by evaluating only certain aspects of the equation for a restricted set of light paths (e.g. direct lighting only), which gave rise to various stylized shading models. Nowadays, with better hardware, the field pursues full accuracy — or, as in this thesis, accuracy and efficiency at the same time.

3. Light Transport in Participating Media

Participating media are substances like fog, water, and smoke. Instead of stopping light at their boundary the way an opaque surface does, they let it travel inside, where it interacts with the many small particles suspended within — water droplets, dust, soot. The interactions are absorption and scattering, just as with surfaces. However, modeling every particle individually would be an enormous labor and would pile even more complexity onto the already hard-to-evaluate rendering equation. Instead, each medium is modeled as a whole — a continuum — carrying a few parameters that determine its nature. The *absorption coefficient* σ_a and the *scattering coefficient* σ_s, both with unit m^{-1}, give the probability density per unit distance traveled that light is absorbed or scattered, respectively; e.g. a larger σ_a makes it more likely that

light is absorbed while traversing the medium. Their sum $\sigma_t = \sigma_a + \sigma_s$ is the *extinction coefficient*, the probability density of *any* interaction, and the ratio $\alpha = \sigma_s/\sigma_t$ — the probability that an interaction scatters rather than absorbs — is called the *scattering albedo*, the medium's counterpart of a surface's albedo.

Four types of interaction are modeled: (1) *emission*: particles generate light energy, so that the medium acts as a light source, e.g. fire; (2) *absorption*: light energy is absorbed by a particle, e.g. in black smoke; (3) *in-scattering*: light traveling in some other direction is scattered into the direction we are following; (4) *out-scattering*: light in our direction loses energy because part of it is scattered into other directions. Note that (3) and (4) both describe the same physical event — a particle redirecting light from one direction into another — and differ only in perspective: watching the old direction, the event is a loss (out-scattering); watching the new one, it is a gain (in-scattering). In summary, when modeling the light at a position inside the medium along one direction, we must account for the gain of energy due to emission at that location and in-scattering from all directions (over the full unit sphere), and the loss of energy due to absorption and out-scattering. This intuition gives rise to the rendering equation for participating media, the *radiative transfer equation* (RTE) [1]. Parametrizing a ray as $\mathbf{x}_t = \mathbf{x} + t\omega$, the change of radiance per unit distance along it is

$$\frac{\mathrm{d}}{\mathrm{d}t} L(\mathbf{x}_t, \omega) = -\sigma_t\, L(\mathbf{x}_t, \omega) + \sigma_a\, L_e(\mathbf{x}_t, \omega) + \sigma_s \int_{\mathcal{S}^2} p(\omega', \omega)\, L(\mathbf{x}_t, \omega')\, \mathrm{d}\omega' \,, \quad \text{(II.6)}$$

Here we write the coefficients without arguments to keep the notation light; in general, each of σ_a, σ_s, σ_t may vary along the ray, taking its value at the current position $\mathbf{x}_t$ (we return to this distinction — homogeneous versus heterogeneous media — at the end of the section). The three terms on the right correspond directly to the interactions above. The first is the loss term: it folds (2) and (4) together, since both remove a fraction $\sigma_t = \sigma_a + \sigma_s$ of the current radiance per unit distance. The second is the gain by (1), the medium's own emitted radiance L_e. The third is the gain by (3): radiance arriving from every direction ω' on the unit sphere $\mathcal{S}^2$, redirected into ω as described by the phase function p (explained below). Note the structural similarity to Eq. (II.4) — an emission term plus a scattering integral over incoming directions — with one key difference: on a surface the interaction happens at a single point, while in a medium it happens continuously, at a rate per unit distance, which is why the RTE is a differential equation.

The loss term alone already explains the most familiar volumetric effect: light dimming as it penetrates a medium. Consider a ray along which we ignore all gains, so that $\frac{\mathrm{d}}{\mathrm{d}t} L = -\sigma_t L$. This first-order ordinary differential equation

has the solution

$$L(\mathbf{x}_t, \omega) = L(\mathbf{x}, \omega)\, e^{-\int_0^t \sigma_t(\mathbf{x}_s)\,\mathrm{d}s}\,, \tag{II.7}$$

i.e. the radiance decays exponentially with the accumulated extinction along the ray (the *optical thickness*). The surviving fraction

$$T_r(\mathbf{x} \to \mathbf{x}_t) = e^{-\int_0^t \sigma_t(\mathbf{x}_s)\,\mathrm{d}s} \tag{II.8}$$

is called the *transmittance* between the two points; the relation is known as the Beer–Lambert law. With the transmittance in hand, the RTE can be integrated along the ray into its integral form, the *volume rendering equation*: the radiance arriving at $\mathbf{x}$ from direction $-\omega$ is the radiance leaving the nearest surface behind the medium (at distance z), attenuated by the transmittance of the full segment, plus the emission and in-scattering gains at every point along the ray, each attenuated by the transmittance from that point forward. Abbreviating the in-scattered radiance from Eq. (II.6) as

$$L_s(\mathbf{x}_t, \omega) = \int_{\mathcal{S}^2} p(\omega', \omega)\, L(\mathbf{x}_t, \omega')\,\mathrm{d}\omega'\,, \tag{II.9}$$

the volume rendering equation reads

$$L(\mathbf{x}, \omega) = T_r(\mathbf{x}_z \to \mathbf{x})\, L_o(\mathbf{x}_z, \omega) + \int_0^z T_r(\mathbf{x}_t \to \mathbf{x}) \left[\sigma_a L_e(\mathbf{x}_t, \omega) + \sigma_s L_s(\mathbf{x}_t, \omega)\right] \mathrm{d}t\,. \tag{II.10}$$

This form is the one a renderer actually evaluates for each camera ray, and the transmittance toward the camera — the camera-side attenuation — is precisely what our method precomputes in Chapter IV.

The phase function $p(\omega', \omega)$ is the medium's relative of the BRDF: where a surface reflects into the hemisphere above it, a particle scatters into the full sphere of directions the light could possibly leave toward. There is a subtle difference in normalization, however. For the BRDF, energy conservation only demands an inequality — $\int_{\mathcal{H}^2} f_r \cos\theta_o\,\mathrm{d}\omega_o \le 1$ — since a surface may absorb part of the incident light, and that absorption is folded into the BRDF's magnitude. The phase function instead integrates to exactly one, $\int_{\mathcal{S}^2} p(\omega', \omega)\,\mathrm{d}\omega = 1$: in a medium, absorption is already handled separately by the split into σ_a and σ_s, so the phase function is left with only one job, distributing the scattered light over the new directions — it is a pure probability distribution. The simplest phase function is the *isotropic* one, $p = \frac{1}{4\pi}$, which scatters equally in all directions — the volumetric counterpart of the Lambertian BRDF. Most real media, however, prefer to scatter forward (e.g. clouds) or backward. The

standard model for such behavior is the *Henyey–Greenstein* phase function [2], which depends only on the angle θ between ω' and ω:

$$p(\theta) = \frac{1}{4\pi} \frac{1 - g^2}{(1 + g^2 - 2g\cos\theta)^{3/2}}\,, \tag{II.11}$$

where the asymmetry parameter $g \in (-1, 1)$ controls the preference: $g > 0$ scatters predominantly forward, $g < 0$ backward, and $g = 0$ recovers the isotropic case.

Finally, a medium is called *homogeneous* when its density is the same everywhere, i.e. the coefficients are constant over the region of the medium; the optical thickness then reduces to $\sigma_t\, t$ and the transmittance to the closed form $e^{-\sigma_t t}$. In a *heterogeneous* medium the coefficients vary with position, and the integral in Eq. (II.8) must itself be evaluated numerically, e.g. by marching along the ray.

4. Monte Carlo Integration

Both rendering equations — the surface form (Eq. (II.4)) and the volume form (Eq. (II.10)) — contain integrals over the incoming directions. The recursion itself is not the obstacle: as noted in Section 2, the incoming radiance inside the integral is the outgoing radiance of some other point, so the equation can be expanded step by step, one scattering event at a time. The obstacle is that each step then still contains an integral over a continuum of directions, whose integrand has no closed form and can only be evaluated point by point — exact evaluation is infeasible even for modern computers. What we can do is *estimate* the integral from a finite number of such point evaluations, which we call *samples*.

The simplest estimate of this kind is the *Riemann sum*: since an integral is the area under the curve, we can cut the domain $[a, b]$ into N strips of equal width and add up the areas of the resulting rectangles,

$$\int_a^b f(x)\,\mathrm{d}x \;\approx\; \frac{b-a}{N} \sum_{k=1}^{N} f(x_k)\,, \tag{II.12}$$

where x_k is a fixed representative of the k-th strip, e.g. its midpoint (Figure II.1, left). As N grows, the strips narrow and the sum converges to the integral.

A small regrouping of the right-hand side of Eq. (II.12),

$$\frac{b-a}{N} \sum_{k=1}^{N} f(x_k) \;=\; (b-a) \cdot \underbrace{\frac{1}{N} \sum_{k=1}^{N} f(x_k)}_{\text{average sample value}}\,,$$

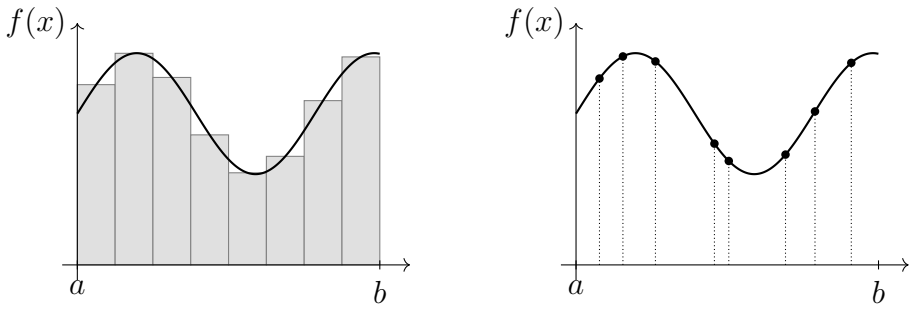

Figure II.1.: Estimating the same integral with $N = 8$ samples. Left: the Riemann sum evaluates f at evenly spaced, pre-determined positions and sums the strip areas. Right: Monte Carlo evaluates f at randomly drawn positions and multiplies the average sample value by the domain size.

reveals a second reading: the second factor is simply the *average* of the sampled function values, so the Riemann sum is "average height times width of the domain". The evenly spaced positions are, however, only one way of taking that average. Suppose we instead throw the N positions onto $[a, b]$ *at random* — uniformly, so that no part of the domain is favored. The mean of the sampled values still approximates the same average height of f: uniform random positions also cover the domain evenly, only in tendency rather than by construction, and their mean therefore settles toward the same value as N grows.

Probability theory makes this precise, and gives the limiting value a name. A random variable X is described by its *probability density function* (PDF) p, where $p(x)\,\mathrm{d}x$ is the probability of X landing in the infinitesimal interval $\mathrm{d}x$ around x. Our uniformly thrown positions make the simplest example: favoring no part of the domain means every point has the same chance of being picked, so p must be a constant over $[a, b]$ — and since the total probability of landing *somewhere* must be one, $\int_a^b p(x)\,\mathrm{d}x = 1$, that constant can only be $\frac{1}{b-a}$. The *expected value* of a function g of X is defined as each possible value weighted by the density of it occurring,

$$\mathrm{E}[g(X)] = \int g(x)\,p(x)\,\mathrm{d}x\,, \tag{II.13}$$

and the *law of large numbers* states that the mean of independent samples, $\frac{1}{N}\sum_k g(X_k)$, converges to $\mathrm{E}[g(X)]$ as $N \to \infty$. The expected value is thus exactly the idealized average that sample means settle toward. For our case $g = f$ and the uniform density, Eq. (II.13) yields $\mathrm{E}[f(X)] = \frac{1}{b-a}\int_a^b f(x)\,\mathrm{d}x$ —

the average height of f over the domain, so that $(b-a)\,\mathrm{E}[f(X)]$ is precisely our integral. Estimating the integral has therefore become: draw uniform samples, take their mean, scale by the domain size. This is *Monte Carlo integration*: where the Riemann sum evaluates f at pre-determined, evenly spaced positions or from a uniform distribution, Monte Carlo draws the positions randomly (Figure II.1, right).

In its general form, the sample positions need not even be uniform. Drawing the X_k from any PDF p that is nonzero wherever f is, the *Monte Carlo estimator*

$$F_N = \frac{1}{N}\sum_{k=1}^{N}\frac{f(X_k)}{p(X_k)} \tag{II.14}$$

still recovers the desired integral. Note that it is not the plain average of the sample values: each $f(X_k)$ is divided by the probability density of drawing that very position, so regions the sampler visits more often than their share are counted with proportionally less weight. This division is exactly what cancels the distortion introduced by non-uniform sampling, as applying Eq. (II.13) with $g = f/p$ shows:

$$\mathrm{E}[F_N] = \frac{1}{N}\sum_{k=1}^{N}\mathrm{E}\left[\frac{f(X_k)}{p(X_k)}\right] = \mathrm{E}\left[\frac{f(X)}{p(X)}\right] = \int\frac{f(x)}{p(x)}\,p(x)\,\mathrm{d}x = \int f(x)\,\mathrm{d}x\,.$$

This holds for any N — the estimator is *unbiased*: its error is never a systematic offset, only random variance around the correct value. (The support condition is what permits the cancellation: wherever $p = 0$, f must vanish too, so no part of the integral is lost.) Choosing the uniform density $p = \frac{1}{b-a}$ recovers the form above.

How does one draw samples from a prescribed density p in the first place, given that a computer only supplies uniform random numbers $\xi \in [0, 1)$? The standard technique is *inversion*, built on the *cumulative distribution function* (CDF)

$$P(x) = \int_{-\infty}^{x} p(x')\,\mathrm{d}x'\,,$$

the total probability of a p-distributed variable falling at or below x. The CDF climbs monotonically from 0 to 1 and determines the distribution completely, since the density is recovered as its derivative, $p = P'$. A variable therefore has density p precisely when its CDF is P, and $X = P^{-1}(\xi)$ satisfies this: by monotonicity, $X \le x$ is equivalent to $\xi \le P(x)$, and a uniform ξ falls below the threshold $P(x)$ with probability exactly $P(x)$. Intuitively, the CDF climbs steeply wherever p is large, so a uniformly thrown ξ is more likely to land on a

steep stretch — and be mapped into the corresponding, high-density region of x.

Why prefer Monte Carlo over the Riemann sum in rendering? The decisive difference is how the two cope with high-dimensional domains. A grid-based rule like the Riemann sum must subdivide every axis of the domain at once: spreading N samples over a d-dimensional grid leaves only $\sqrt[d]{N}$ subdivisions per axis, so a million samples in six dimensions amount to a grid of merely ten strips in each direction — and every further dimension coarsens the grid again. This rapid loss of resolution is known as the *curse of dimensionality*. Monte Carlo builds no grid at all: samples are thrown into the domain individually, and the estimate sharpens steadily with their number, regardless of the dimension. The integrals of light transport are of exactly this hostile kind: expanding the recursion of the rendering equation adds one integration domain per scattering event, so the full problem is an integral over paths of arbitrary length — of unbounded dimension. In addition, the integrand jumps discontinuously (an emitter is visible from one direction and occluded from a neighboring one), which grid-based rules handle poorly but point sampling tolerates without complaint. The price Monte Carlo pays is variance, which in an image shows up as pixel noise that fades only slowly with more samples.

The freedom to choose p in Eq. (II.14) is also the main lever for reducing that variance: the closer p is proportional to the integrand, the less the individual terms $f(X_k)/p(X_k)$ fluctuate — if $p \propto f$ exactly, every sample yields the same value and the variance is zero. Placing samples preferentially where the integrand is large is called *importance sampling* [9]. Renderers importance-sample whatever factors of the integrand they know in closed form: the BRDF lobe and the phase function (Henyey–Greenstein admits an analytic inversion). Participating media add one more integral to estimate: the volume rendering equation (Eq. (II.10)) accumulates contributions from every point along the ray through the medium; a sample from this domain is a distance t along the ray — physically, the position of the light's next interaction with the medium. Because the integrand carries the transmittance, the distance should be drawn with density $p(t) = \sigma_t e^{-\sigma_t t}$ (for a homogeneous medium). This density can be read in two ways. Physically, the light first survives to depth t undisturbed, with probability $e^{-\sigma_t t}$ by Eq. (II.8), and then interacts within the following sliver $\mathrm{d}t$, with probability $\sigma_t \,\mathrm{d}t$ — σ_t being the interaction probability per unit distance. Formally, the prefactor σ_t is exactly the constant that normalizes the exponential to total probability one over $[0, \infty)$: in an unbounded medium, the light is certain to interact somewhere. Inverting the CDF $P(t) = 1 - e^{-\sigma_t t}$

gives

$$t = -\frac{\ln(1-\xi)}{\sigma_t}\,, \tag{II.15}$$

known as *free-flight* (or distance) sampling. It is a routine used in nearly every volumetric light transport algorithm, including the photon tracing pass of our method.

Assembling these pieces yields *path tracing*, the canonical Monte Carlo renderer [5]: starting from the eye, repeatedly importance-sample one direction (and, in media, one distance) at each interaction, building a random path that is terminated into a light source; averaging many such paths per pixel gives an unbiased estimate of Eq. (II.4) and (II.10). Path tracing converges to the exact solution and serves as the reference in Chapter V, but participating media make it slow: light may scatter many times inside a medium before escaping, and paths that undergo specular reflection or refraction rarely manage to connect to a light source at all — the caustic problem from Section 2, aggravated in volumes. These weaknesses motivate the two-pass methods of the next section.

5. Density Estimation and Photon Mapping

Path tracing, as described in the previous section, traces rays from the camera backwards into the scene and collects radiance when a path hits an area light — or, more efficiently, at every bounce by attempting a direct connection to a light source, a technique known as next event estimation [7]. One class of light paths, however, resists this approach: *caustics*, paths that leave the light, pass through one or more specular reflections or refractions, bounce off a diffuse surface, and end in the eye — the bright patterns at the bottom of a swimming pool, or the focused streak behind a glass. At a specular surface the next ray direction is fully determined by the incoming one (e.g. a mirror), so a path traced from the eye side cannot steer itself toward the light; it must hit it by chance. For a point light behind an ideal specular chain that chance is exactly zero, and for a small area light vanishingly small. In a path-traced image, caustics therefore appear noisy and take a long time to even emerge.

Focusing on caustics, one arrives at a reversal: why not trace paths from the *light* first, determine where its energy lands throughout the scene, and only then render the image using this already-known information? This is the idea behind photon mapping [3] and the family of related methods covered in Chapter III. Of a light source we know its shape, position, orientation, and its power Φ in Watts. Power, as introduced in Section 1, is a property of the source as a whole: every point of an area light emits toward every direction,

and following all of these positions and directions is as impossible as evaluating an integral exactly. Monte Carlo integration therefore enters once more, this time on the emission side: we draw a starting position on the light's surface and an outgoing direction at random and follow the resulting path through the scene. Each such path is one sample in the sense of Eq. (II.14), and the quantity being sampled is the flux: the N traced paths divide the finite power Φ among themselves, each carrying a packet $\Delta\Phi \approx \Phi/N$ (in general weighted by the inverse of its sampling densities — the familiar f/p pattern).

Each packet is traced through the scene like any light path, until it lands on a diffuse surface. Connecting it from there to the camera runs into the mirror image of the caustic problem: a light path can no more aim at a pinhole camera than an eye path can aim at a point light. So instead of forcing the connection, we deposit the energy where it landed: at every diffuse surface the path crosses, a *photon* is stored — position, incoming direction ω_j, and carried flux $\Delta\Phi_j$ — and the collection of all stored photons is called the *photon map*. The connection to the camera is postponed to the rendering pass.

That connection cannot be exact either. A camera ray's hit point and a stored photon's position are both produced by independent random sampling, and as with any continuous distribution, the probability of two sampled points coinciding is zero. What a hit point $\mathbf{x}$ *can* know is the photons in its neighborhood; estimating a continuous quantity from the discrete samples scattered around a query point is a classic statistical technique called *density estimation* [8]. To see what exactly is being estimated, we follow the derivation of [3] starting from the scattering integral of Eq. (II.4). Solving the definition of radiance (Eq. (II.3)) for the incident flux gives

$$L_i(\mathbf{x}, \omega_i) \cos\theta_i = \frac{\mathrm{d}^2\Phi_i(\mathbf{x}, \omega_i)}{\mathrm{d}A\, \mathrm{d}\omega_i} .$$

Substituting this into the scattering integral cancels both the cosine and the solid angle:

$$L_r(\mathbf{x}, \omega_o) = \int_{\mathcal{H}^2} f_r(\mathbf{x}, \omega_i, \omega_o) \frac{\mathrm{d}^2\Phi_i(\mathbf{x}, \omega_i)}{\mathrm{d}A\, \mathrm{d}\omega_i} \mathrm{d}\omega_i = \int_{\mathcal{H}^2} f_r(\mathbf{x}, \omega_i, \omega_o) \frac{\mathrm{d}^2\Phi_i(\mathbf{x}, \omega_i)}{\mathrm{d}A} . \tag{II.16}$$

The reflected radiance is an integral of the BRDF against incoming flux per area — and the photons are exactly discrete packets of incoming flux. Replacing the continuous flux $\mathrm{d}^2\Phi_i$ by the k packets $\Delta\Phi_j$ found within a disc of radius r around $\mathbf{x}$, and the area $\mathrm{d}A$ by the disc's area, gives the *radiance estimate*

$$L_r(\mathbf{x}, \omega_o) \;\approx\; \sum_{j=1}^{k} f_r(\mathbf{x}, \omega_j, \omega_o) \frac{\Delta\Phi_j}{\pi r^2} . \tag{II.17}$$

The sum over photons stands in for the integral over incoming directions, with the photons' arrival directions ω_j as the samples; no explicit $\frac{1}{N}$ or sampling density appears because both were already folded into each packet during the light pass. The division by πr^2 turns gathered flux into flux per area, and the BRDF, evaluated once per photon, is what finally connects each light-side direction ω_j to the camera-side direction ω_o — the gather is the meeting point of the two passes.

Eq. (II.17) uses the simplest possible kernel: every photon within distance r counts fully, every photon beyond it not at all. Whatever the kernel, the estimate treats all gathered photons as if they had arrived exactly at $\mathbf{x}$, so any detail of the true illumination finer than r — the sharp edge of a caustic, a shadow boundary — is averaged away: the result is *blurred*. This is a different kind of error from Monte Carlo noise: for a fixed radius the estimate is systematically offset, i.e. *biased*, no matter how many photons are traced. In exchange, it is far smoother than a path-traced estimate from the same amount of work. The bias is not fatal: the estimator is *consistent*, meaning that as the number of photons grows and the radius shrinks accordingly, the estimate converges to the true value. Biased but smooth versus unbiased but noisy is the essential trade-off between photon mapping and path tracing.

Inside a participating medium the same story repeats. A camera path entering a medium is scattered at random distances into random directions, so reaching a small light by chance is again hopeless — and *volumetric caustics*, light focused by specular surfaces into shafts inside the medium, suffer both problems at once. The light pass extends naturally: propagation distances are drawn by free-flight sampling (Eq. (II.15)), and a photon is stored at every scattering event inside the medium, forming a volume photon map [4]. The gathered quantity is now the in-scattered radiance L_s of Eq. (II.9), and the neighborhood becomes a sphere:

$$L_s(\mathbf{x}, \omega) \;\approx\; \frac{1}{\sigma_s} \sum_{j=1}^{k} p(\omega_j, \omega)\, \frac{\Delta\Phi_j}{\frac{4}{3}\pi r^3}\,. \tag{II.18}$$

The phase function takes the place of the BRDF, the sphere's volume that of the disc's area, and the leading $\frac{1}{\sigma_s}$ compensates for the photons having been stored in proportion to the medium's scattering strength — it cancels against the σ_s that multiplies L_s in Eq. (II.10).

Photon mapping thus splits rendering into two passes — trace flux from the lights and store it, then estimate radiance from the stored density during rendering — and this two-pass structure is the foundation our method rests on. How the stored representation evolved from points to beams, how the blur

can be driven to zero progressively, and how gathering can be replaced by its scatter-side counterpart, splatting, is the subject of Chapter III.

6. Ray Tracing and Rasterization

The previous sections spoke repeatedly of tracing rays — following light, or the eye's gaze, through the scene to collect what we are after, radiance. The mechanism underneath all these algorithms is *ray tracing*. A ray is defined by an origin and a direction: an arrow placed at a certain location in the scene, facing a certain way. To determine where the arrow lands, we compute its intersections with the objects of the scene: each object is described by an explicit or implicit function and is, like the ray, a set of positions, so intersecting the two sets yields the positions they share, and the one closest along the ray is where it lands. Computing intersections efficiently is a field of research in its own right; the standard remedy is an acceleration structure such as the bounding volume hierarchy, which encloses geometry in nested boxes so that a ray missing a box skips everything inside it at once [6] — there is no reason to test objects the ray cannot hit e.g. behind the camera. In these terms, path tracing reads: for every pixel and every sample on the film, spawn a ray, trace it through the scene, and gather contributions along the way.

The other paradigm for forming an image is *rasterization*, and it inverts the question of ray tracing: instead of spawning a ray from each pixel to determine what that pixel sees, it projects the geometry onto the screen to determine which pixels each object covers. The scene's surfaces — in practice, triangles — pass through a fixed pipeline: each vertex is transformed into the camera's perspective (in a programmable vertex shader), the hardware rasterizer determines which pixels the projected triangle covers and emits a *fragment* for each of them — a pixel-sized candidate piece of the triangle, carrying its interpolated depth; a second programmable stage (the fragment shader) computes a color for every fragment, and a *depth test* resolves which fragment is actually seen: every pixel remembers the depth of the nearest fragment drawn onto it so far, and a new fragment is kept only when it is closer than that — whatever lies farther is hidden. This keep-the-nearest behavior suits opaque surfaces. For content that does not fully hide what lies behind it — glass, smoke, a volume — the pipeline offers a different mode as an addition: *blending*, in which a new fragment's color is combined with the color already accumulated at the pixel, e.g. added on top of it, instead of replacing it. Our method relies on this mode in Chapter IV to sum up the light contributions rasterized into each pixel.

A single projection, however, answers only what is *directly* visible: it is equivalent to the first bounce of an eye path, and nothing more. Effects that require the path to continue — mirror reflection, refraction, soft shadows, accurate indirect lighting — therefore lie beyond plain rasterization and call for extra setup of their own. What rasterization gives up in generality, it returns in speed. Its workload consists of the same small programs applied to millions of mutually independent items — vertices in the one stage, covered pixels in the other — and graphics hardware is built precisely to run such uniform, independent work massively in parallel. This is what makes rasterization fast enough for real-time use, e.g. in games.

The two paradigms are not mutually exclusive: a single renderer may use both, choosing one or the other for each part of its work. Our method is such a hybrid — how the work is divided between the two is the subject of Chapter IV, and earlier methods that attempted similar combinations are reviewed in Chapter III.

Chapter III.

Related Work

1. Classic Photon Mapping

1.1. Participating media consideration

2. Progressive Photon Mapping

3. Camera Beam Gathering Photon (Jarosz)

4. Jarosz's framework

4.1. Photon Beam

4.2. Generalization of different estimators

4.3. Beam x Beam 1D estimator

5. Photon Splatting

6. Photon Disk Splatting for Participating media

7. Relationship to 3D Gaussian Splatting

Chapter IV.

Method

Chapter V.

Results

Chapter VI.

Discussion and Future Work

Chapter VII.

Conclusion

Bibliography

[1] Subrahmanyan Chandrasekhar. *Radiative Transfer*. Oxford University Press, 1950.

[2] Louis G. Henyey and Jesse L. Greenstein. "Diffuse radiation in the galaxy". In: *Astrophysical Journal* 93 (1941), pp. 70–83. DOI: 10.1086/144246.

[3] Henrik Wann Jensen. "Global illumination using photon maps". In: *Rendering Techniques '96 (Proceedings of the 7th Eurographics Workshop on Rendering)*. Springer, 1996, pp. 21–30. DOI: 10.1007/978-3-7091-7484-5_3.

[4] Henrik Wann Jensen and Per H. Christensen. "Efficient simulation of light transport in scenes with participating media using photon maps". In: *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '98)*. 1998, pp. 311–320. DOI: 10.1145/280814.280925.

[5] James T. Kajiya. "The rendering equation". In: *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '86)*. 1986, pp. 143–150. DOI: 10.1145/15922.15902.

[6] Daniel Meister et al. "A survey on bounding volume hierarchies for ray tracing". In: *Computer Graphics Forum* 40.2 (2021), pp. 683–712. DOI: 10.1111/cgf.142662.

[7] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. 4th. MIT Press, 2023.

[8] Bernard W. Silverman. *Density Estimation for Statistics and Data Analysis*. Chapman & Hall, 1986.

[9] Eric Veach. "Robust Monte Carlo Methods for Light Transport Simulation". PhD thesis. Stanford University, 1997.

Appendix A.

An Appendix